

10.5. Шаблоны. Классы структур данных

Независимость от типов данных

Любая структура данных и алгоритмы имеют дополнительную ценность, если они могут хранить и работать с данными различных типов. Такая универсальность (или что одно и то же, независимость от данных) может быть достигнута в Си++ различными способами:

- в обычном Си (см. 9.3) «переход от типа к типу» и абстрагирование от конкретного хранимого типа возможно на основе преобразования типов указателей и использования указателя **void***, олицетворяющего «память вообще». Совместно с механизмом динамического связывания возможно создание алгоритмов, в которых фрагмент, ответственный за работу с конкретным типом данных, передается через *указатель на функцию*;
- в Си++ на основе **полиморфизма** (виртуальных функций) (см.11.4) возможно создание интерфейсных классов, способных объединять единым механизмом доступа различные классы. Если эти классы являются «обертками» известных типов данных, то независимость от типов хранимых данных можно обеспечить ссылками на интерфейсный класс.

Однако приведенные выше способы не обеспечивают **синтаксической** совместимости, т.е. они реализуются как технологические приемы, а не как элементы языка. По аналогии с переопределением операций (см. 10.3) хотелось бы использование естественного синтаксиса, где вместо **int**, **double** и т.п. фигурировало бы абстрактное обозначение типа, например, **T**.

Такое средство, позволяющее создавать заготовку функции или целого класса, в котором вместо конкретного имени типа данных будет фигурировать его символическое обозначение, называется **шаблоном**. В первом приближении смоделировать шаблон в обычном Си можно с использованием директив препроцессора для подстановки имен – **define**. Обозначив именами **T** и **N** тип и размерность массива, можно создать класс с использованием этих имен везде, где это необходимо.

```
//-----105-01.cpp
// Имитация шаблона в обычном Си
#define T int           // Параметры заданы через подстановку имен
#define N 100          // Тип элементов и размерность массива
struct Array{
    T Data[N];
    int k;              // Текущее кол-во элементов
    Array(){ k=0; }     // Конструктор - массив пуст
    void add(T &v){      // Добавление элемента
        if (k>=N) return;
        Data[k++]=v;}
    T remove(int m){    // Удаление элемента по номеру
        T foo;
        if (m>=k) return foo;
        foo=Data[m];
        for(int i=m;i<k-1;i++)
            Data[i]=Data[i+1];
        k--; return foo; }
};
void main(){
    Array A;
    int B[10]={6,2,8,3,56,7,89,5,7,9};
```

```
for (int i=0;i<10;i++) A.add(B[i]);
cout << A.remove(2) << endl; }
```

Чтобы применить ее для другого типа данных, нужно отредактировать директиву **define**, заменив, например, имя **int** на **double**. Но все же основным недостатком этой модели является однократность ее использования. В Си++ текстовая заготовка класса позволяет из одного описания создавать множество классов, отличающихся типом используемых объектов и размерностями данных.

Итак, шаблон можно определить как *текстовую заготовку определения класса с параметром, обозначающим тип используемых внутри переменных*. Сразу же, не дожидаясь описания синтаксиса, отметим особенности трансляции шаблона:

- шаблон является описанием *группы классов*, отличающихся используемым типом данных;
- при создании (определении) объекта шаблонного класса указывается тип данных, для которого он создается;
- при определении объекта шаблонного класса конкретный тип подставляется в шаблон вместо параметра и создается текстовая заготовка *экземпляра класса*, которая транслируется и дает собственный оригинальный программный код;
- и только затем происходит трансляция определения самого объекта.

Самое главное, в отличие от обычных объектов, объект шаблонного класса требует отдельного экземпляра класса для того типа, который обозначен в объекте.

Замечание: при чтении транслятором шаблона заголовка класса и шаблонов встроенных в него функций и методов их трансляция **не производится**. Это происходит в другое время: при трансляции определения объекта шаблонного класса генерируется текстовая заготовка экземпляра класса. Отсюда некоторый нюанс:

- чтобы проверить, как транслируется шаблон, нужно описать хотя бы один объект шаблонного класса;
- весь шаблон, в том числе и шаблоны методов, следует размещать в заголовочном файле проекта;
- на каждый тип данных – параметр шаблона создается отдельный класс и в сегменте команд размещается программный код его методов.

Следующим примером попробуем «убить двух зайцев». Во-первых, пояснить довольно витиеватый синтаксис шаблона, а во-вторых, выделить особенности реализации структур данных и использованием технологии ООП. Основной принцип шаблона, добавление к имени класса «довеска» в виде имени – параметра (например, **vector<T>**). Это имя обозначает внутренний тип данных, который может использоваться в любом месте класса: как указатель, ссылка, формальный параметр, результат, локальная или статическая переменная. Во всем остальном шаблон не отличается от обычного класса. Само имя шаблона (**vector**) теперь обозначает не один класс, а группу классов, отличающихся внутренним типом данных.

```
//-----105-02.cpp
//---- Шаблон СД - динамический массив указателей
template <class T> class vector{
    int sz,n;           // Размерность ДМУ и кол-ко элементов
    T **obj;            // Массив указателей на параметризованные
public:                 // объекты типа T
    T *operator[](int); // оператор [int] возвращает указатель на
```

```

// параметризованный объект типа T
operator int();           // Возвращает текущее количество указателей
int append(T*);           // Добавление указателя на объект типа T
int index(T*);            // Поиск индекса хранимого объекта
vector(int);              // Конструктор
~vector(){ delete []obj; } // Деструктор
};

```

Данный шаблон может использоваться для порождения объектов-векторов, каждый из которых хранит указатели объекты определенного типа. Имя класса при этом составляется из имени шаблона **vector** и имени типа данных (класса), который подставляется вместо параметра **T**.

```

vector<int>    a;
vector<double> b;
extern class   time;
vector<time>   c;

```

При определении каждого вектора с новым типом объектов транслятором генерируется описание нового класса по заданному шаблону (естественно, неявно в процессе трансляции). Например, для типа **int** транслятор получит.

```

class vector<int>{
    int sz,n;           // Размерность ДМУ и кол-ко элементов
    int **obj;          // Массив указателей на параметризованные
public:                // объекты типа T
    int *operator[](int); // оператор [int] возвращает указатель на
                        // параметризованный объект типа T
    operator int();      // Возвращает текущее количество указателей
    int append(int *);   // Добавление указателя на объект типа T
    int index(int *);    // Поиск индекса хранимого объекта
    vector(int);         // Конструктор
    ~vector(){ delete []obj; } // Деструктор
};

```

Обратите внимание, что это *иллюстрация* принципа подстановки, а не фрагмент программы с синтаксисом Си++. Далее следует утверждение типа «масло масляное»: встроенные функции (методы) шаблонного класса – есть шаблонные функции. Это означает, что методы класса, включенные в шаблон, также должны «заготовками» с тем же самым параметром, то есть генерироваться для каждого нового типа данных. То же самое касается переопределяемых операторов.

```

//-----105-02.cpp
template <class T> vector<T>::operator int()
{ return n; }
template <class T> T* vector<T>::operator[](int k){
    return k>=n ? NULL : obj[k]; }
template <class T> int vector<T>::index(T *pobj){
    for ( int i=0; i<n; i++)
        if (pobj == obj[i]) return i;
    return -1;}
template <class T> vector<T>::vector(int sz0){
    sz=sz0; n=0; obj=new T*[sz]; }

template <class T> int vector<T>::append(T *pobj){
    if (n>=sz) return 0;
    obj[n++]=pobj;
    return 1;}

```

Приведенный пример касается только методов, «вынесенных» из заголовка класса. Для каждого из них пишется отдельный шаблон, а сам класс фигурирует в нем

под именем вида **vector<T>**. Возможность непосредственного определения методов в заголовке шаблонного класса (inline-методов) также остается.

Шаблоны могут иметь также и параметры-константы, которые используются для статического определения размерностей внутренних структур данных. Кроме того, шаблон может использоваться для размещения не только указателей на параметризованные объекты, но и самих объектов. В качестве примера рассмотрим шаблон для построения циклической очереди ограниченного размера на основе статического массива (см. 6.1), хранящей непосредственно сами объекты (значения, а не указатели).

```
//-----105-03.cpp
//----- Шаблон с параметром-константой
template <class T,int size> class FIFO{
    int fst,lst;                // Индексы начала и конца очереди
    T queue[size];              // Массив объектов класса T размерности size
public:
    T from();                   // Функции включения-исключения
    int into(T);                //
    FIFO(){ fst=lst=0; }        // Конструктор
};
template <class T,int size> T FIFO<T,size>::from(){
    T work=0;
    if (fst !=lst){
        work = queue[lst++];
        if (lst==size) lst=0;
    }
    return work;}
template <class T,int size> int FIFO<T,size>::into(T obj) {
    if ((fst+1)%size==lst)
        return 0;              // Проверка на переполнение
    queue[fst++] = obj;
    if (fst==size) fst=0;
    return 1; }
```

Объекты такого шаблонного класса при определении имеют два параметра: тип данных и константу – статическую размерность.

```
struct x {...};
FIFO<double,100>    a;
FIFO<int,20>        b;
FIFO<x,50>          c;
```

Особенности разработки шаблонов структур данных

Шаблон является исключительно средством подстановки, он ничего не меняет в ни в существе класса структуры данных, для которой строится шаблон, ни в принципах передачи объектов класса – параметров шаблона. Во-первых, это касается способа хранения объектов в структуре данных: она может содержать указатели, а может и сами объекты (копии, значения):

- если шаблон хранит указатели на объекты, то он не касается проблем корректного копирования объектов и «не отвечает» за их создание и уничтожение. Деструктор шаблона обязан уничтожить динамические компоненты структуры данных (динамические массивы указателей, элементы списка), но он обычно не уничтожает хранимые объекты;
- если шаблон хранит сами объекты, то он «должен быть уверен» в корректном копировании объектов при их записи и чтении из структуры

данных (конструктор копирования о переопределение присваивания для объектов, содержащих динамические данные). При разрушении структуры данных разрушаются и копии хранимых в ней объектов.

Во-вторых, шаблон может использовать ряд стандартных операций по отношению к типу данных – параметру шаблона: *присваивание (копирование), сравнение*, а также *ввод и вывод* в стандартные потоки. При использовании шаблона с параметром – не базовым типом, а классом, необходимо следить, чтобы эти операции были в нем переопределены.

Шаблоны классов списков

Для представления списка или дерева необходимы две сущности: элементы списка (вершины дерева), связанные между собой и заголовок – указатель на первый элемент списка (корневую вершину). В технологии ООП есть два равноправных решения:

- разрабатывается один класс, объекты которого играют разную роль в процессе работы класса. Первый объект – заголовок, создается программой (статически или динамически), доступен извне и не содержит данных (по крайней мере, в момент конструирования). Остальные объекты, содержащие данные, создаются динамически методами, работающими с первым объектом. Этот вариант имеет некоторые тонкости, связанные с тем, что программа должна уметь различать, где она работает с элементом списка, а где с элементом-заголовком;
- разрабатывается два класса – класс элементов списка и класс списка как такового, содержащего, как минимум, его заголовок. Объекты первого (вспомогательного) класса пользователем (программой, работающей с классом) не создаются. Они все – динамические, и их порождают методы второго (основного) класса;

Рассмотрим первый вариант на примере шаблона односвязного списка. Коллизий, связанных с распознаванием заголовочного элемента и элемента, содержащего данные, не возникает. Все методы, описанные в заголовке класса, применяются только по отношению к заголовочному элементу (в контексте все время фигурирует установка на первый элемент списка в виде **p=next**, либо установка на «предыдущий»-заголовочный в виде **p=this**). Часть простых методов реализована в самом заголовке класса, в них для указателей на элементы списка используется сокращенный контекст вида **list *p**, хотя класс **list** является шаблонным.

```
//-----105-04.cpp
template <class T> class list{
    list<T> *next;           // Указатель на следующий в списке
    T data;                  // Элемент списка хранит сам объект
    list(T& v) { data=v; next=NULL;} // Скрытый конструктор для элементов с данными
public: list() { next=NULL; }  // Конструктор для элемента - заголовка
    ~list(){                  // Деструктор рекурсивный
        if (next!=NULL) delete next;
    }                        // рекурсивное удаление следующего
    void front(T& v){         // Включение в начало
        list *q=new list(v);
        q->next=next; next=q;
    }
    void end(T &v){           // Включение в конец
        list *p,*q=new list(v); // Дойти до последнего
```

```

        for(p=this;p->next!=NULL;p=p->next);
        p->next=q;           // Следующий за последним - новый
    }
    void insert(T&,int);      // Включение по логическому номеру
    void insert(T&);         // Включение с сохранением порядка
    T remove(int);           // Извлечение по ЛН
    operator int(){           // Приведение к int = размер списка
        list *q; int n;
        for(q=next,n=0; q!=NULL; n++,q=q->next);
        return n;
    }
    friend ostream &operator<<(ostream&, list<T>&);
    friend istream &operator>>(istream&, list<T>&);
};                             // Переопределенный вывод в поток

```

Единственная коллизия возникает в конструкторе и деструкторе. Для заголовочного элемента используется конструктор без параметров. Методы списка, создавая динамические элементы – объекты того же класса, используют закрытый конструктор с параметром – значением, хранимым в элементе списка. В методе удаления элемента из списка кроме логического исключения выбранного элемента из цепочки у него обнуляется указатель на следующего (**p->next=NULL**). Это делается для того, чтобы исключить рекурсивный вызов деструктора при удалении одного элемента списка.

Специфика односвязного списка проявляется в реализации методов вставки по номеру, с сохранением порядка и удаления. Текущий указатель в цикле ссылается на предыдущий элемент списка по отношению к искомому (поэтому, например, используется выражение **q->next->data** для сравнения значения в искомом элементе).

```

//-----105-04.cpp
template <class T> void list<T>::insert(T &newdata,int n){
    list<T> *p,*q;
    p=new list<T>(newdata);
    for (q=this; q->next!=NULL && n!=0; n--,q=q->next);
    p->next=q->next;           // Вставка после текущего
    q->next=p; }               // Отсчет от заголовка
//-----
template <class T> void list<T>::insert(T &newdata){
    list<T> *p,*q;
    p=new list<T>(newdata);           // Сравнивать СО СЛЕДУЮЩИМ
    for (q=this; q->next!=NULL && newdata>q->next->data; q=q->next);
    p->next =q->next;                 // Вставка ПОСЛЕ текущего
    q->next=p;}
//-----
template <class T> T list<T>::remove(int n){
    list<T> *q,*p;           // Указатель на ПРЕДЫДУЩИЙ
    for (q=this; q->next!=NULL && n!=0; n--,q=q->next);
    T val;
    if (q->next==NULL) return val; // Такого нет
    val=q->next->data;
    p=q->next; q->next=p->next;    // Исключить СЛЕДУЮЩИЙ из цепочки
    p->next=NULL;                 // Для исключения рекурсивного деструктора
    delete p;
    return val;}

```

Переопределенные операции ввода-вывода в стандартные потоки поддерживают саморазворачивающийся симметричный формат представления данных, использующий счетчик элементов. При вводе в соответствии с прочитанным значением счетчика многократно читается объект класса **T**, значение которого каждый раз включается в список.

```

//-----105-04.cpp
template <class T> ostream &operator<<(ostream &O, list<T> &R){

```

```

list<T> *q;
O << (int)R << endl;
for (q=R.next; q!=NULL; q=q->next) O << q->data << endl;
return O; }
template <class T> istream &operator>>(ostream &l, list<T> &R){
    T val; int n;
    l >> n;
    while(n--!=0){ l >> val; R.front(val); }
    return l; }

```

Второй вариант с разнесением элементов и заголовка в отдельные классы рассмотрим на примере циклического списка. Класс элемента списка **elem** определяется внутри закрытой части шаблонного класса **list**, поэтому его полное имя будет весьма витиеватым **list<T>::elem**. Если его методы выносить из заголовка класса, то они получат имена вида **list<T>::elem::before** (на память неизбежно приходит Остап-Сулейман-Берта-Мария-Бендер Бей или в коротком контексте Остап Бендер). В остальных случаях внутри класса **list** транслятор в состоянии распознать короткое имя типа **elem** вместо длинного.

```

//-----105-05.cpp
template <class T> class list {
    class elem{                                // Внутренний класс -
    public:                                    // элемент списка

        T *pd;
        elem *next,*prev;                    // Указатели
        int remove(){                        // Метод - выкуси себя сам
            if (next==this)                  // Единственный - ничего
                return 1;                    // не делать
            next->prev=prev;                 // Иначе - перекинуть
            prev->next=next;                 // указатели соседей
            return 0;}                      // друг на друга

        void before(elem *p){                // Включить себя перед p
            next=p;                          // Следующий за собой - p
            prev=p->prev;                    // Предыдущий - p->prev
            p->prev->next=this;               // Следующий за p->prev - сам
            p->prev=this;                    // Предыдущий для p - сам
        }                                    // Конструктор

        elem(T *s) {pd=s; next=prev=this; }
        ~elem() {}                          // Деструктор пустой
    }; //----- окончание объявления elem

```

Элемент циклического списка является более «интеллектуальным», его связи позволяют ему без посторонней помощи включить и исключить себя в заданное место цепочки элементов. Это же позволяет сделать синтаксис методов более компактным, используя контекст текущего элемента списка.

```

//-----продолжение заголовка класса list-----105-05.cpp
    elem *head;                                // Заголовок списка
public:
    void end(T *s){                            // Включение в конец
        if (head==NULL) head=new elem(s);
        else (new elem(s))->before(head);
    }                                           // Не пустой - перед первым
    void front(T *s){                          // В начало = в конец
        end(s); head=head->prev;               // + перемещение заголовка
    }
    void sort();
    operator int();                            // Приведение к int - длина списка
    friend ostream &operator<<(ostream&,list&);
    list() { head=NULL; }
    ~list();

```

Стиль программирования класса списка можно сделать более компактным, используя уже написанные методы. Например, для включения элемента в конец непустого списка достаточно создать динамический объект – элемент списка и тут же вызвать для него метод включения перед первым (т.е. после последнего) **new elem(s)** - **>before(head)**. В остальном методы шаблонного класса используют стандартные приемы работы с циклическими списками.

```
//-----105-05.cpp
template <class T> list<T>::~~list(){ // Деструктор списка
    int k;
    do { // Цикл, пока не остался последний
        elem *q=head->prev; // Последний
        k=q->remove(); // исключить
        delete q; // и удалить
    } while(k==0);} // пока он не единственный
template <class T> list<T>::operator int(){
    if (head==NULL) return 0; // Приведение к int - длина списка
    list<T>::elem *p; int n; // Обход списка, начиная со второго
    for (p=head->next,n=1; p!=head;n++,p=p->next);
    return n; }
template <class T> ostream &operator<<(ostream &O,list<T> &L){
    int n=L; // Приведение к int - длина списка
    list<T>::elem *p=L.head;
    O << n << endl;
    while(n--!=0){ // Просмотр списка и вывод
        O << *p->pd << endl; // хранимых объектов
        p=p->next; }
    return O; }
template <class T> istream &operator>>(istream &I,list<T> &L){
    int n; I >> n; // Ввод из потока - чтение счетчика
    T val; //
    while(n--!=0){I >> val; L.end(val); }
}
```

Замечание: переопределенный оператор ввода из потока, определенный вне класса, использует только открытые методы класса **list**, поэтому нет необходимости объявлять его дружественным.

Сортировка списка вставками имеет некоторые особенности. Хотя во время сортировки создается второй список, но для этого используется отдельный указатель **elem***, а не объект класса **list**. В процессе сортировки элементы списка не уничтожаются, а просто исключаются из исходного и вставляются в выходной с сохранением порядка. По окончании сортировки указатель на выходной список переносится в текущий объект.

```
//-----105-05.cpp
template <class T> void list<T>::sort(){ // Сортировка вставками
    if (head==NULL || head->next==head) return;
    elem *q,*out,*p; int k; // Длина списка 0 или 1 - выход
    q=head->prev; // Создать из последнего элемента
    q->remove(); // выходной список
    q->next=q->prev=q; out=q;
    do { q=head->prev; // Удалить последний элемент
        k=q->remove(); p=out;
        do{ // Просмотр выходного, пока
            if (*q->pd < *p->pd) break;
            p=p->next; // не встретит больше себя
        } while(p!=out); // Или пока список не кончится
        q->before(p); // Вставка нового перед найденным
        if (*q->pd < *out->pd) // Если перед первым - коррекция заголовка
            out=q;
    } while(k==0);
    head=out;} // Вернуть новый список на старое место
```


Шаблоны классов деревьев

При программировании деревьев в технологии ООП рассмотренные выше технологические приемы получают свое дальнейшее развитие. Дело том, что рекурсивные алгоритмы, работающие с вершинами дерева, могут быть реализованы только в классе вершины. Текущий объект является текущей вершиной, с которой работает рекурсивный алгоритм, а рекурсивный вызов – это вызов того же самого метода для вершины – потомка. В редких случаях, когда имеет место жадный выбор очередного потомка, линейная рекурсия может быть реализована в цикле в классе самого дерева.

Посмотрим, как будет выглядеть двоичное дерево (см. 8.5), реализованное в виде шаблона в системе классов – вершина дерева (**node**) и дерево (**btree**). В классе вершины реализуются все рекурсивные алгоритмы. Даже там, где имеет место линейная рекурсия, поскольку контекст текущей вершины позволяет «экономить синтаксис» и записывать выражения для доступа к вершинам в более короткой форме. Так, рекурсивный вызов метода **get** для левого потомка имеет вид **l->get()**.

В технологии ООП есть еще один нюанс. Любой метод можно вызвать только для какого-либо объекта, т.е. существующей вершины. Поэтому при включении в дерево новых вершин нельзя передавать нулевые ссылки на вершины и переходить по ним, как это делалось в традиционной технологии. Нужно сначала убедиться, что указатель на объект-потомок не равен **NULL**, либо создать такой объект, а уже потом вызывать рекурсивный метод.

```
//-----105-06.cpp
template <class T>
class btree{
    class node{                // Внутренний класс -
    public: T *pd;              // вершина дерева
        int cnt;               // Счетчик вершин в поддереве
        node *l,*r;           // Указатели на потомков
    T *get(T *s){              // Поиск по значению (двоичный)
        if (*s==*pd) return pd;
        if (*s<*pd) return l==NULL ? NULL : l->get(s);
        else return r==NULL ? NULL : r->get(s); }
    T *get(int m){              // Извлечение по логическому номеру
        if (m>=cnt) return NULL;
        if (l!=NULL){          // Если есть левое поддерево и туда попадаем
            if (m<l->cnt) return l->get(m);
            m-= l->cnt;          // Не попадаем - отсчитать число вершин в нем
        }                      // ЛН=0 - текущая вершина
        if (m--==0) return pd;
        return r->get(m);        // Иначе - с остатком в правое поддерево
    }
    void insert(T *s){          // Включение в дерево
        cnt++;
        if (*s<*pd){
            if (l==NULL) l=new node(s);
            else l->insert(s); }
        else {
            if (r==NULL) r=new node(s);
            else r->insert(s);}
    }
    node(T *s) {cnt=1; pd=s; l=r=NULL; }
    ~node(){                    // Деструктор рекурсивный для поддерева
        if (l!=NULL) delete l;
        if (r!=NULL) delete r; }
```

В каждой вершине имеется счетчик потомков, который позволяет сделать извлечение по логическому номеру в виде линейно-рекурсивного алгоритма (см. 8.5). Он же позволяет при выводе в текстовый поток сохранять не только хранимые данные, но и информацию о структуре дерева (см. 8.2), что приводит при вводе из текстового потока к восстановлению первоначальной структуры дерева. Такой формат, очевидно, предназначен для сохранения дерева в текстовом файле, а не для просмотра его содержимого.

```
//-----105-06.cpp
friend ostream &operator<<(ostream &O,node &N){
    O << N.cnt << endl << *N.pd << endl;           // Сохранить счетчик вершин и данные
    if (N.l!=NULL) O << 0 << endl; else O << *N.l; // Операция для левого объекта
    if (N.r!=NULL) O << 0 << endl; else O << *N.r; // Операция для правого объекта
    return O; }
friend istream &operator>>(istream &O,node &N){
    N.pd=new T; O >> *N.pd;
    int n; O >> n;
    if (n!=0) { N.l=new node(NULL);                // Создать вершину
                N.l->cnt=n; O >> *N.l; }           // Вызвать для нее ту же операцию
    l >> n;
    if (n!=0) { N.r=new node(NULL); N.r->cnt=n; O >> *N.r; }
    return O; }
}; //----- Окончание класса node
```

Здесь, конечно же имеются «архитектурные излишества». Совсем не обязательно в классе **node** использовать синтаксис переопределенных операций ввода/вывода в стандартные потоки. Но уж если это делается, то в качестве операндов в них фигурируют объекты-потомки в виде ***N.l**. Опять же, при вводе сначала создается объект-потомок (если его счетчик потомков не равен 0), а затем он становится операндом в переопределяемой операции для текущего потока (рекурсивное переопределение операции).

Сам класс дерева является интерфейсным. Он содержит указатель на корневую вершину дерева и дублирует все рекурсивные методы класса вершин. Практически везде он проверяет наличие корневой вершины, при отсутствии – создает ее, а при наличии – вызывает для нее аналогичный рекурсивный метод в ее классе.

```
//-----105-06.cpp
node *head;
public:
    btree(){ head=NULL; }
    ~btree(){ if (head!=NULL) delete head; }
    btree &operator<<(T *s){ // Вставка в дерево – переопределение <<
        if (head==NULL) head=new node(s);
        else head->insert(s);
        return *this; }
    T *operator[](int m){ // Извлечение по ЛН – переопределение []
        return head==NULL ? NULL : head->get(m); }
    T *operator[](T *key){ // Поиск по значению – переопределение []
        return head==NULL ? NULL : head->get(key); }
    operator int(){ // Приведение к int – количество вершин
        return head==NULL ? 0 : head->cnt; }
    friend ostream &operator<<(ostream &O, btree &T){
        O << (T.head==NULL ? 0 : *T.head);
        return O; } // Вывод в поток
    friend istream &operator>>(istream &O, btree &T){
        if (T.head!=NULL) { delete T.head; T.head=NULL; }
        int n; O >> n; // Ввод из потока – разрушить старое
        if (n!=NULL){ // Создать вершину и для нее вызов той же операции
            T.head=new node(NULL); T.head->cnt=n; O >> *T.head; }
        return O; };
```

Немного об STL

Шаблоны позволяют структурам данным абстрагироваться от их содержимого. А есть ли возможность алгоритмам абстрагироваться от структур данных. Такая возможность появляется, если для шаблонных классов структур данных создать классы объектов-движков, в функции которых входит позиционирование и доступ к хранимым элементам структур данных (аналоги операции $i=0$, $A[i]=v$, $i++$, $i--$ в массиве). Такие классы называют **итераторами**, они устанавливают стандартный синтаксис доступа к элементам структуры данных, через который разработанный алгоритм может быть применен к любой из них.

Лабораторный практикум

Разработать шаблон класса структуры данных с параметром – типом хранимых объектов **T**, включая «джентльменский набор» операций: конструктор «пустой» структуры данных, деструктор, операции добавления, включения и исключения по логическому номеру, сортировки и включения с сохранением порядка (при наличии в **T** стандартной операции сравнения). Разработать также методы сохранения и загрузки структуры данных из стандартного текстового потока при наличии переопределенных операций $<<$ и $>>$ для потока и объектов хранимого класса. Загрузку структуры данных из стандартного потока производить в создаваемые для этой цели динамические объекты. В качестве вариантов использовать любые известные структуры данных.

Вопросы без ответов

Определить вид структуры данных, создаваемой шаблоном, а также значения всех переменных после выполнения **main**.

```
//-----105-07.cpp
//-----1
template <class T> class S{
    T *A; int n,sp;
public: S(int n0){ n=n0; sp=-1; A=new T[n]; }
        S& operator <<(T m){
            if (sp!=n-1) A[++sp]=m;
            return *this; }
        S& operator >>(T &m){
            if (sp==n-1) m=0; else m=A[sp--];
            return *this; };
void main(){
    S<int> a(10);  a << 1 << 5 << 7;
    int a1,a2,a3,a4=6; a >> a1 >> a2 >> a3 >> a4;}
//-----2
template <class T,int n> class S{
    T A[n]; int sp;
public: S(){ sp=-1; }
        S& operator <<(T m){
            if (sp!=n-1) A[++sp]=m;
            return *this; }
        S& operator >>(T &m){
            if (sp==n-1) m=0; else m=A[sp--];
            return *this; };
void main(){
    S<int,3> a;  a << 1 << 5 << 7 << 9;
    int a1,a2,a3,a4=6; a >> a1 >> a2 >> a3 >> a4;}
//-----3
template <class T> class S{
    T **A; int n,sp;
public: S(int n0){ n=n0; sp=-1; A=new T*[n]; }
        S& operator <<(T &v){
```

```

        if (sp!=n-1) A[++sp]=&v;
        return *this; }
T *operator ()(){
    T *m=NULL;
    if (sp!=-1) m=A[sp--];
    return m; };
void main(){
    S<int> a(3); int V[]={1,3,5,7};
    a << V[1] <<V[3] << V[0] << V[2];
    int a1,a2,*a3,*a4; a1=*a(); a2=*a(); a3=a(); a4=a(); }
//-----4
template <class T,int n> class S{
    int A[n]; int k;
public: S(){ k=0; }
    S& operator <<(T &m){
        if (k!=n) A[k++]=m;
        return *this; }
    S& operator >>(T &m){
        if (k==0) return *this;
        m=A[0];
        for (int i=0; i<k-1; i++) A[i]=A[i+1];
        k--; return *this; };
void main(){
    S<int,10> a; a << 1 << 5 << 7;
    int a1,a2,a3,a4=6; a >> a1 >> a2 >> a3 >> a4; }
//-----5
template <class T> class S{
    T *A; int n,k;
public: S(int n0){ n=n0; k=0; A=new T[n]; }
    S& operator <<(T m){
        if (k!=n) A[k++]=m;
        return *this; }
    T operator [] (int i)
        { if (k==0 || i>=k) return 0;
        return A[i];}
    S& operator -(int m){
        if (k==0 || m>=k) return *this;
        for (int i=m; i<k-1; i++) A[i]=A[i+1];
        k--; return *this; };
void main(){
    S<int> a(10); a << 1 << 5 << 7 << 9 << 11;
    int a1=a[2],a2,a3; a - 2 - 2; a2=a[2]; a3=a[5]; }
//-----6
template <class T, int n> class S{
    T A[n]; int k;
public: S(){ k=0; }
    S& operator <<(T m){
        if (k!=n) A[k++]=m;
        return *this; }
    T operator [] (int i)
        { if (k==0 || i>=k) return 0;
        return A[i];}
    S& operator -(int m){
        if (k==0 || m>=k) return *this;
        for (int i=m; i<k-1; i++) A[i]=A[i+1];
        k--; return *this; };
void main(){
    S<int,10> a; a << 1 << 5 << 7 << 9 << 11;
    int a1=a[2],a2,a3; a - 2 - 2; a2=a[2]; a3=a[5]; }
//-----7
template <class T, int n> class S{
    T *A[n]; int k;
public: S(){ k=0; }
    S& operator <<(T &m){

```

```

        if (k!=n) A[k++]=&m;
        return *this; }
T *operator[](int i)
{ if (k==0 || i>=k) return NULL;
  return A[i];}
S& operator -(int m){
  if (k==0 || m>=k) return *this;
  for (int i=m; i<k-1; i++) A[i]=A[i+1];
  k--; return *this; };

void main(){
  int b[]={1,5,7,9,11}; S<int,10> a;
  for (int i=4; i>=0; i--) a << b[i];
  int a1=*a[2],a2,a3,*p; a - 2 - 2; a2=*a[2]; p=a[5]; }
//-----8
template <class T> class S{
  T *A; int n,k;
public: S(int n0){ n=n0; k=0; A=new T[n]; }
  S& operator <<(T m){
    int i,j;
    for (i=0; i<k && m > A[i]; i++);
    for (j=k-1; j>=i; j--) A[j+1]=A[j];
    A[i]=m; k++;
    return *this; }
  T operator[](int i)
    { if (k==0 || i>=k) return 0;
      return A[i];};

void main(){
  S<int> a(10); a << 13 << 5 << 17 << 9 << 11;
  int a1=a[1],a2,a3; a2=a[2]; a3=a[5]; }
//-----9
template <class T, int n> class S{
  T A[n]; int k;
public: S(){ k=0; }
  S& operator <<(T m){
    int i,j;
    for (i=0; i<k && m > A[i]; i++);
    for (j=k-1; j>=i; j--) A[j+1]=A[j];
    A[i]=m; k++;
    return *this; }
  T operator[](int i)
    { if (k==0 || i>=k) return 0;
      return A[i];}
};

void main(){
  S<int,10> a; a << 13 << 5 << 17 << 9 << 11;
  int a1=a[1],a2,a3; a2=a[2]; a3=a[5]; }
//-----10
template <class T, int n> class S{
  T *A[n]; int k;
public: S(){ k=0; }
  S& operator <<(T &m){
    int i,j;
    for (i=0; i<k && m > *A[i]; i++);
    for (j=k-1; j>=i; j--) A[j+1]=A[j];
    A[i]=&m; k++;
    return *this; }
  T *operator[](int i)
    { if (k==0 || i>=k) return NULL;
      return A[i];};

void main(){
  int b[]={13,5,17,9,11};
  S<int,10> a;
  for (int i=0;i<5;i++) a << b[i];
  int a1=*a[1],a2,*p; a2=*a[2]; p=a[5]; }

```